Self Joins

==========

can you perform a join with a single table?

```
CREATE TABLE employees (
        employee_id INT PRIMARY KEY,
        employee_name VARCHAR(100),
        department_id INT,
        manager_id INT,
        salary DECIMAL(10, 2)
);

INSERT INTO employees (employee_id, employee_name, department_id, manager_id, salary)
VALUES

(1, 'Rajesh Kumar', 1, NULL, 200000.00),          -- CEO, No manager
(2, 'Sonal Gupta', 2, 1, 150000.00),              -- VP IT, reports to Rajesh Kumar
(3, 'Anita Desai', 2, 2, 120000.00),              -- Manager IT, reports to Sonal Gupta
(4, 'Vikram Singh', 3, 1, 110000.00),             -- VP HR, reports to Rajesh Kumar
(5, 'Priya Nair', 2, 3, 130000.00),               -- Developer IT, reports to Anita Desai
(6, 'Amit Sharma', 3, 4, 100000.00),              -- HR Manager, reports to Vikram Singh
(7, 'Neha Patil', 4, 1, 140000.00),               -- VP Finance, reports to Rajesh Kumar
(8, 'Rohit Verma', 4, 7, 90000.00),               -- Finance Analyst, reports to Neha Patil
(9, 'Kavita Joshi', 2, 3, 125000.00),             -- Developer IT, reports to Anita Desai
(10, 'Manish Kulkarni', 5, 1, 145000.00),         -- VP Marketing, reports to Rajesh Kumar
(11, 'Nidhi Agarwal', 5, 10, 95000.00),           -- Marketing Manager, reports to Manish Kulkarni
(12, 'Suresh Iyer', 4, 7, 85000.00),              -- Accountant, reports to Neha Patil
(13, 'Ravi Menon', 6, 1, 135000.00),              -- VP Operations, reports to Rajesh Kumar
(14, 'Anjali Sinha', 6, 13, 105000.00),           -- Operations Manager, reports to Ravi Menon
(15, 'Tarun Mehta', 2, 2, 115000.00),             -- IT Support, reports to Sonal Gupta
(16, 'Deepika Rao', 3, 4, 98000.00),              -- HR Assistant, reports to Vikram Singh
(17, 'Karan Chawla', 5, 10, 92000.00),            -- Marketing Analyst, reports to Manish Kulkarni
(18, 'Meena Kapoor', 4, 7, 102000.00),            -- Finance Manager, reports to Neha Patil
(19, 'Sanjay Reddy', 6, 13, 108000.00),           -- Logistics Coordinator, reports to Ravi Menon
(20, 'Shweta Tiwari', 2, 3, 127000.00);           -- Developer IT, reports to Anita Desai


select * from employees;
select * from employees;

select e1.employee_name as employee,
e1.salary as employee_salary,
e2.employee_name as manager,
e2.salary as manager_salary
from employees e1
left join employees e2
on e1.manager_id = e2.employee_id;
```

employees manager left join

-- all employees who have no manager

```
select e1.employee_name as employee,
e1.salary as employee_salary,
e2.employee_name as manager,
e2.salary as manager_salary
from employees e1
left join employees e2
on e1.manager_id = e2.employee_id
where e2.employee_id is null;
```

employees manager right outer join on the left side we get null

-- all employees who do not manage anyone

```
select e1.employee_name as employee,
e1.salary as employee_salary,
e2.employee_name as manager,
e2.salary as manager_salary
from employees e1
right join employees e2
on e1.manager_id = e2.employee_id
where e1.manager_id is null;
```

-- employees earning more than their managers

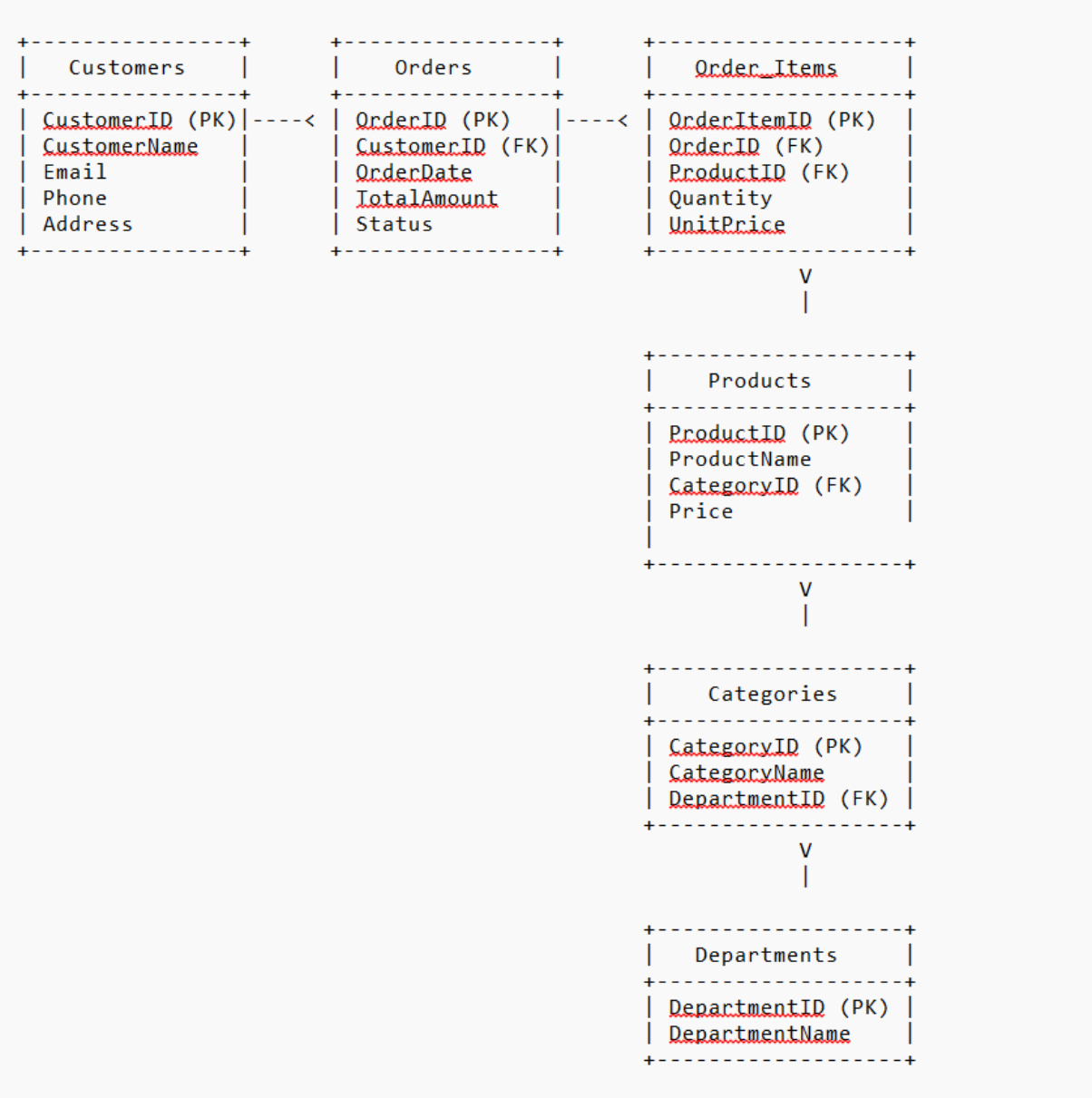
```
select e1.employee_name as employee,
e1.salary as employee_salary,
e2.employee_name as manager,
e2.salary as manager_salary
from employees e1
inner join employees e2
on e1.manager_id = e2.employee_id
where e1.salary > e2.salary;
```

-- employees earning more than the avg salary of their department

```
select e.employee_id,
e.employee_name,
e.department_id,
e.salary,
avg_salaries.avg_salary
from employees e
inner join
(select department_id, avg(salary) as avg_salary
from employees
group by department_id) avg_salaries
on e.department_id = avg_salaries.department_id
and e.salary > avg_salaries.avg_salary;
```

```
CREATE TABLE courses (
        course_id INT PRIMARY KEY,
        course_name VARCHAR(100),
        prerequisite_id INT,
        duration_weeks INT
);

INSERT INTO courses (course_id, course_name, prerequisite_id, duration_weeks) VALUES
(1, 'Intro to Programming', NULL, 10),
(2, 'Data Structures', 1, 8),
(3, 'Algorithms', 2, 10),
(4, 'Operating Systems', 2, 12),
(5, 'Databases', 1, 8);

-- find the courses that take longer to complete than their prerequisite

courses prerequisites

select c1.course_name as course,
c1.duration_weeks as course_duration,
c2.course_name as prerequisite,
c2.duration_weeks as prerequisite_duration
from courses c1
inner join courses c2
on c1.prerequisite_id = c2.course_id
where c1.duration_weeks > c2.duration_weeks ;

===============
```

Joining more than 2 tables
==========



select * from orders;
select * from customers;
select * from order_items;
select * from products;
select * from categories;
select * from departments;

```
select c.customer_id,
c.customer_fname,
c.customer_lname,
c.customer_city,
c.customer_state,
c.customer_zipcode,
o.order_id,
o.order_date,
o.order_status,
oi.order_item_id,
oi.order_item_quantity,
oi.order_item_subtotal,
oi.order_item_product_price,
p.product_id,
p.product_name,
p.product_price,
cat.category_id,
cat.category_name,
d.department_id,
d.department_name
from customers c
left join orders o
on c.customer_id = o.order_customer_id
join order_items oi
on o.order_id = oi.order_item_order_id
join products p
on oi.order_item_product_id = p.product_id
join categories cat
on p.product_category_id = cat.category_id
join departments d
on cat.category_department_id = d.department_id;
```